

Time Series

Data Science

Time Series

- forecasting
- autocorrelation
- ARMA models
- time-series features
- deep learning for time series
- anomaly detection

Temporal Dependence

Time series observations are not independent.

Instead, present values depend on the past: $y_t \sim f(y_{t-1}, y_{t-2}, \dots)$

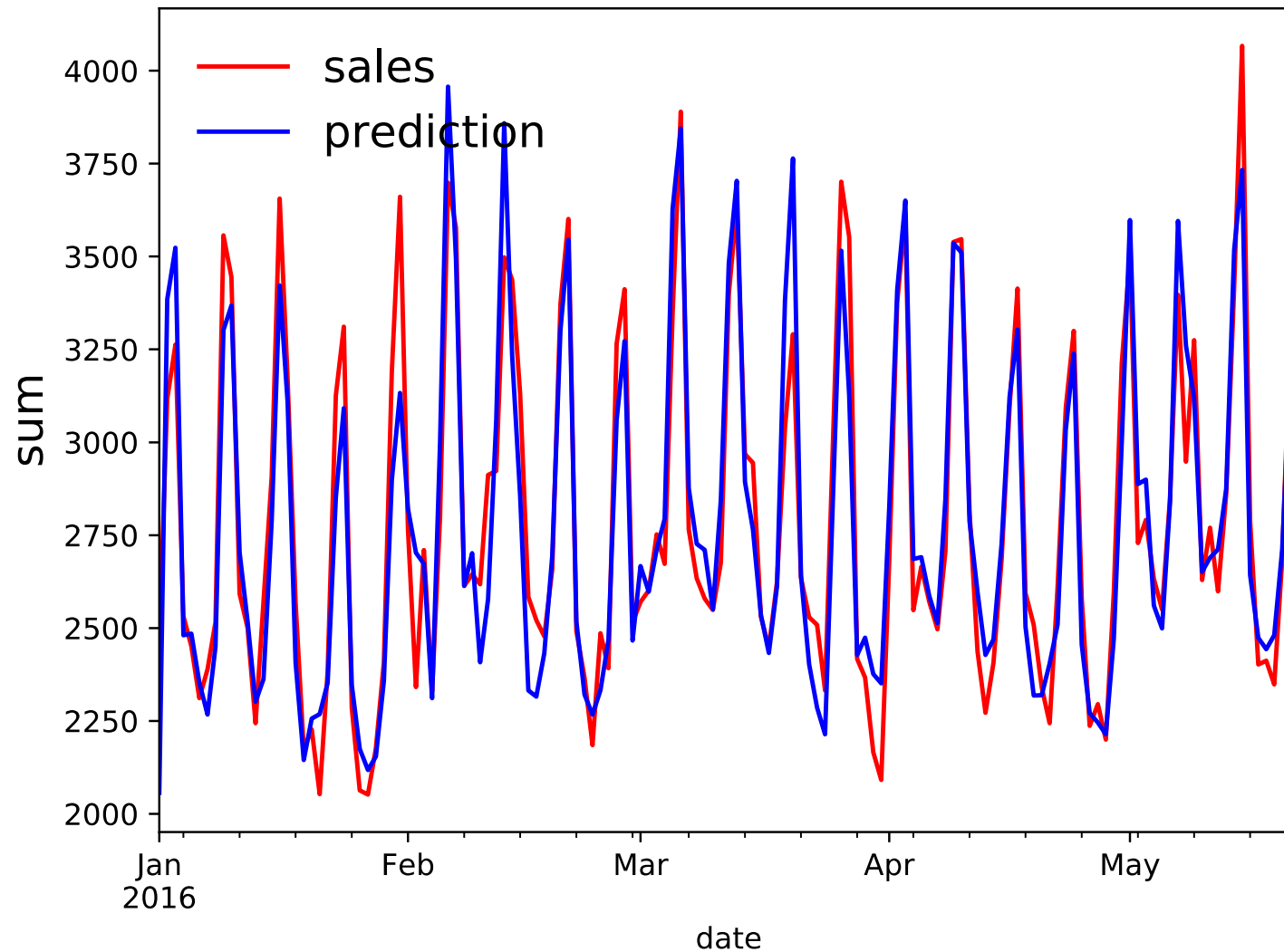
examples: weather, stock prices, sensor measurements, patient trajectories

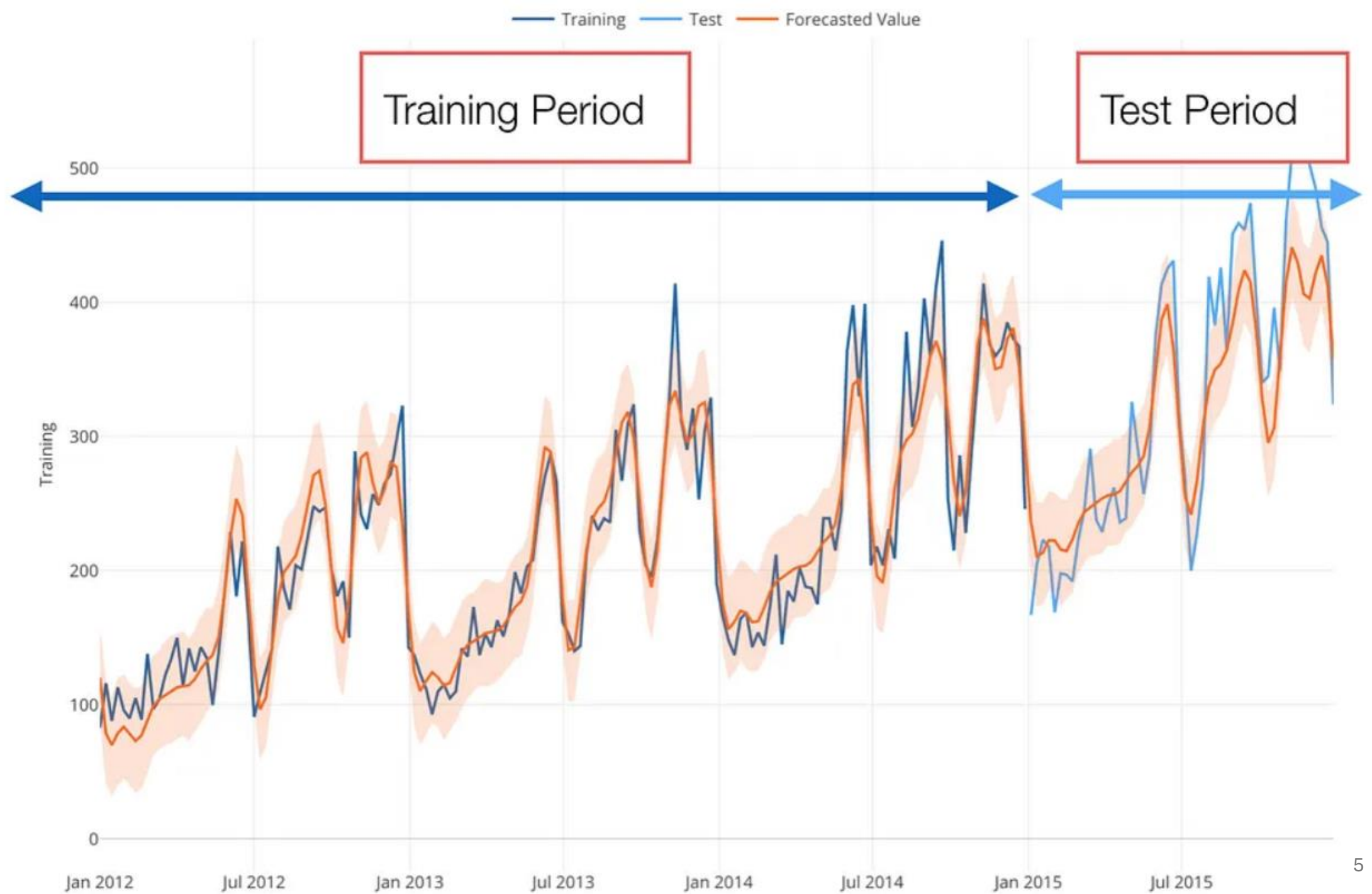
common temporal patterns:

- trend: long-term change
- seasonality: repeating cycles
- lag dependence: memory of past states

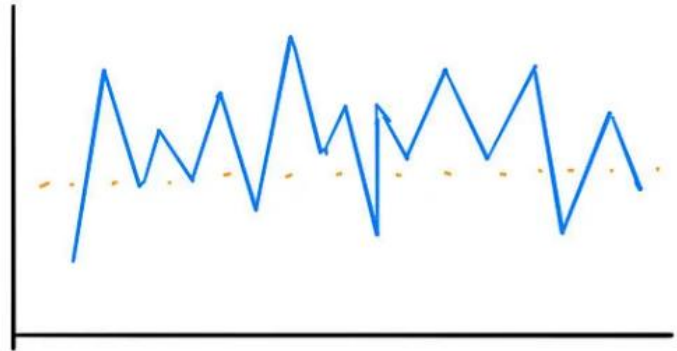
ignoring time structure → weaker models, misleading evaluation

Example: Forecasting



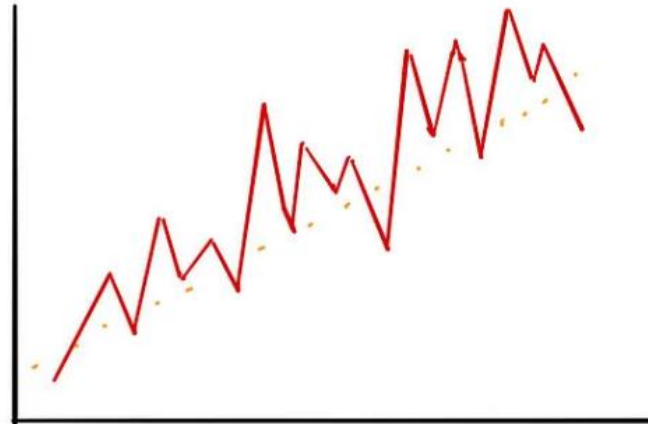


Statistical Patterns



STATIONARY

TREND



SEASONAL

Autocorrelation (ACF) and Partial Autocorrelation (PACF)

ACF: correlation with past values

$$\rho_k = \text{Corr}(y_t, y_{t-k})$$

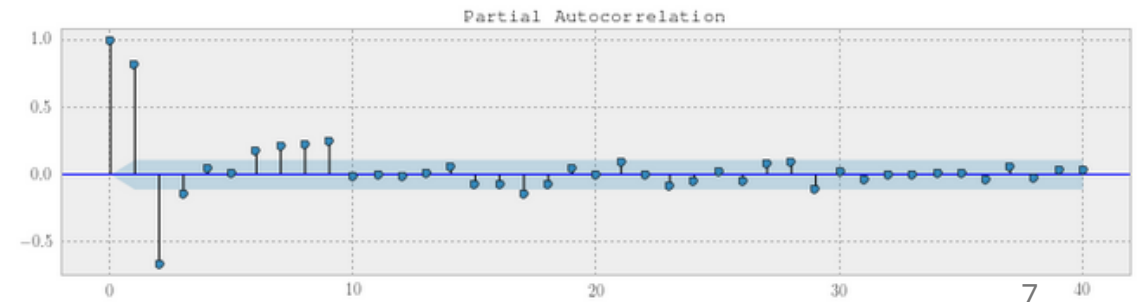
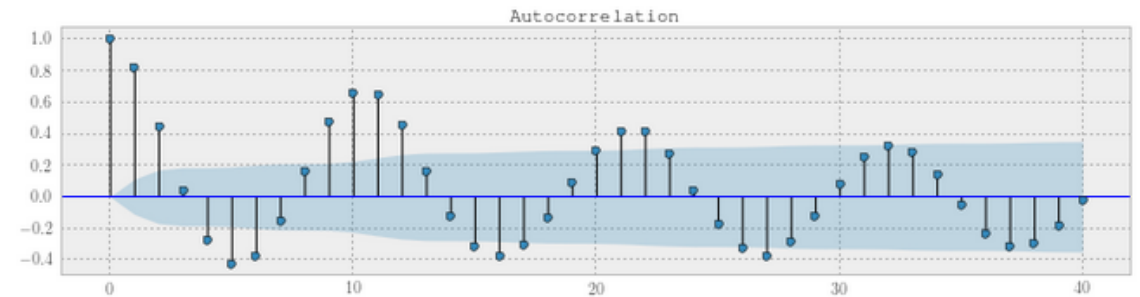
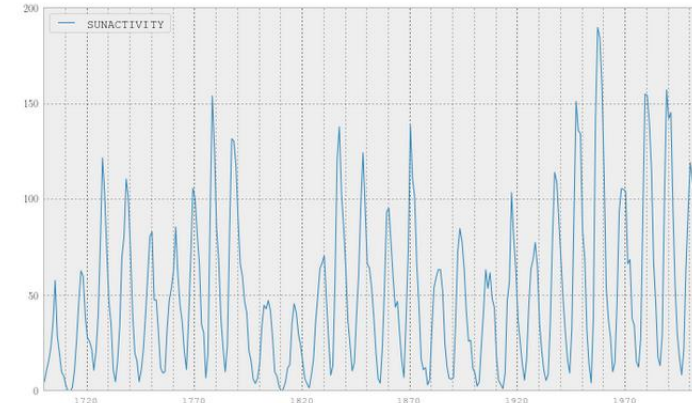
“How much does the past resemble the present?”

→ persistence, seasonality, moving-average structure

PACF: direct effect of lag k , controlling for shorter lags

$$\text{Corr}(y_t, y_{t-k} | y_{t-1}, \dots, y_{t-k+1})$$

→ autoregressive order, direct temporal dependencies



ARMA

AutoRegressive (AR) model:

predict from past observations (lagged values)

$$y_t = c + \phi_1 y_{t-1} + \dots + \phi_p y_{t-p} + \epsilon_t$$

Moving Average (MA) model:

predict from past errors (lagged shocks/residuals)

$$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \dots + \theta_q \epsilon_{t-q}$$

$AR(p) + MA(q) \rightarrow ARMA(p,q)$ $\rightarrow$ decide on p and q by ACF/PCF

ARIMA

idea: extend ARMA to handle non-stationary time series

remove trend by differencing: $y'_t = y_t - y_{t-1}$  $d = 1$: linear trend
 $d = 2$: $y'_t = y_t - 2 \cdot y_{t-1} + y_{t-2}$
→ include integrated component: $AR(p) + I(d) + MA(q) \rightarrow ARIMA(p,q)$

additional extensions:

- SARIMA (Seasonal ARIMA): adds seasonal autoregressive, differencing, and moving-average terms
- SARIMAX (SARIMA + eXogenous variables): includes external information (e.g. weather, holidays, promotions)

key assumption: after differencing (and seasonal adjustment, if needed), the series should be approximately stationary

Stationarity

mean, variance, autocorrelation structure remain stable over time

important for ARMA assumptions, easier generalization, stable relationships

common fixes:

- differencing
- log transform (converts multiplicative relationships into additive ones, forcing variations to remain consistent throughout the entire timeline)
- seasonal adjustment

Types of Time Series Forecasting Information

auto-correlation:

- using past values of considered time series as function of lag
- identifying periodicity (seasonality) and trend

→ ARMA

exogenous variables:

- using correlations of other variables to considered time series
- examples in demand forecasting: price, weather
- such effects lasting for some time create spurious auto-correlation

ML-based Forecasting

decomposition-based approaches:

additive components such as trend, seasonality, holidays

→ additive regression / generalized additive modeling

example method: prophet

natural way for multivariate model: training one model combining several independent time series, distinguished by additional feature (e.g., product ID)

Feature Engineering for Time Series

idea: convert temporal structure into input features for ML models

- lag features (past values): captures autoregressive behavior
- calendar/time features: captures seasonality and recurring patterns
- rolling & aggregated features: captures local trend and variability
- external/exogenous features: incorporates outside influences

Lag Features & Rolling Statistics

create features from past values $x_{t-1}, x_{t-2}, x_{t-3}, \dots$

example: predict today's sales using sales yesterday, sales last week, ...

→ captures short- and long-term patterns

compute aggregation statistics over a window (e.g., rolling mean, rolling sum)

example: average sales over last 7 days

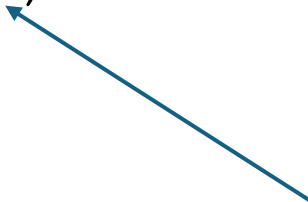
Exponential Smoothing

exponentially weighted moving average:

$$\hat{y}_t = \alpha \cdot y_{t-1} + (1 - \alpha) \cdot \hat{y}_{t-1}$$

```
X.sort_values(['date'])  
.groupby(['store', 'item'])['y']  
.apply(lambda x: x.shift(3))  
.ewm(alpha=0.15).mean()
```

potentially further grouped
by important lag (like day of
week) or exogenous
variables (like promotion)



prediction horizon

Trend & Seasonality

capture long-term movement such as increasing/decreasing trend
difference between current and past values

capture repeating patterns:

- day of week
- month
- holiday indicators

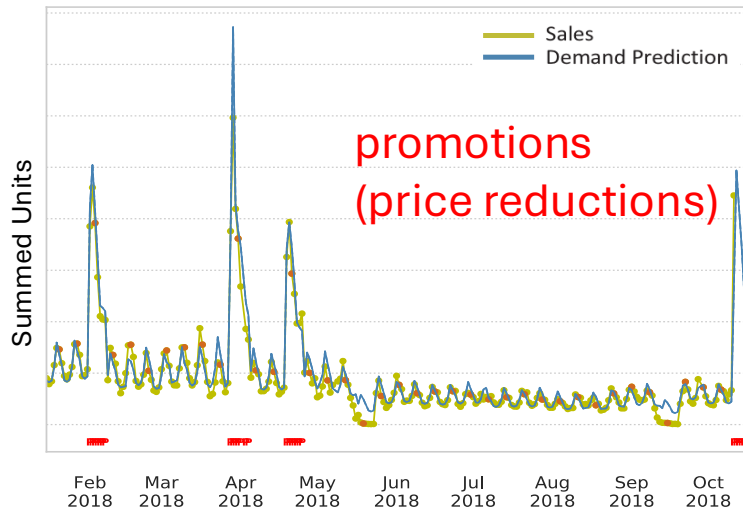
→ often critical in time series problems

Demand Forecasting with ML

many individual time series to consider

typical retail grocery chain:

- products (items): ~20k
- locations (stores): ~500
- daily/hourly aggregated sales



advantages of ML over traditional univariate time series forecasting:

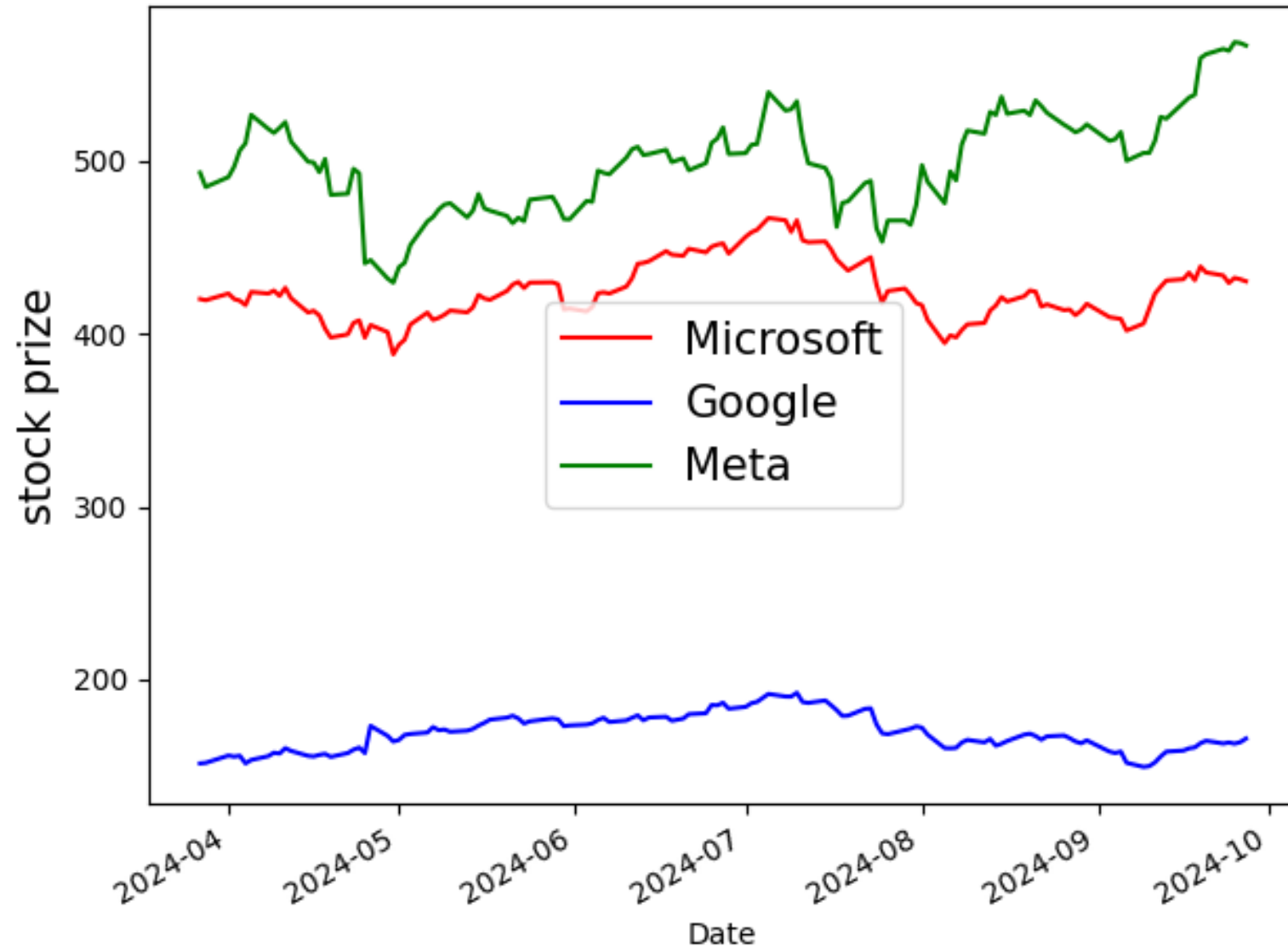
combined learning on all time series of product-location combinations (rather than separately optimizing individual time series)
→ **reduces variance** by exploiting commonalities

natural consideration of many exogenous variables (prices, promotions, holidays, weather, ...)
→ **reduces bias**

to be noted:

- categorical features important (products and locations → high cardinality)
- mainly multiplicative effects
- demand (approximately) following Poisson (or rather negative binomial) distribution

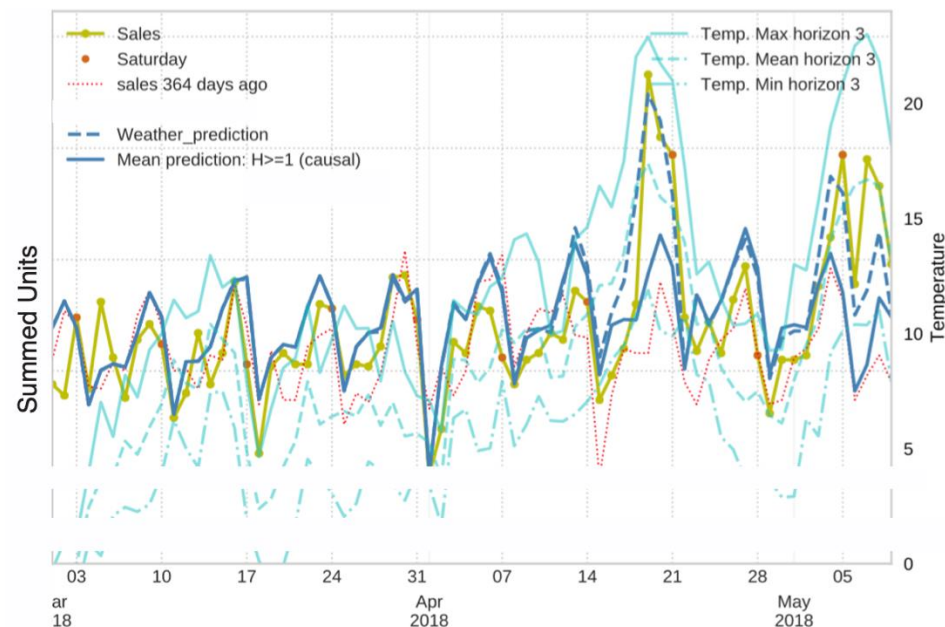
You can decrease uncertainty by looking at similar time series



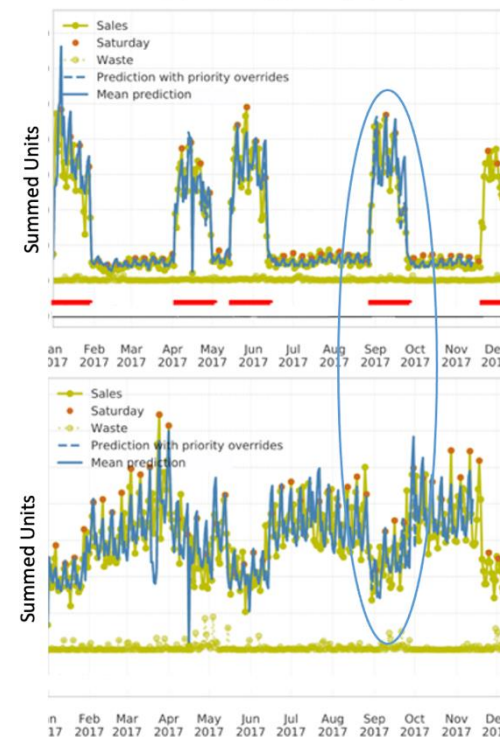
Example: Retail Features

Summer Weather

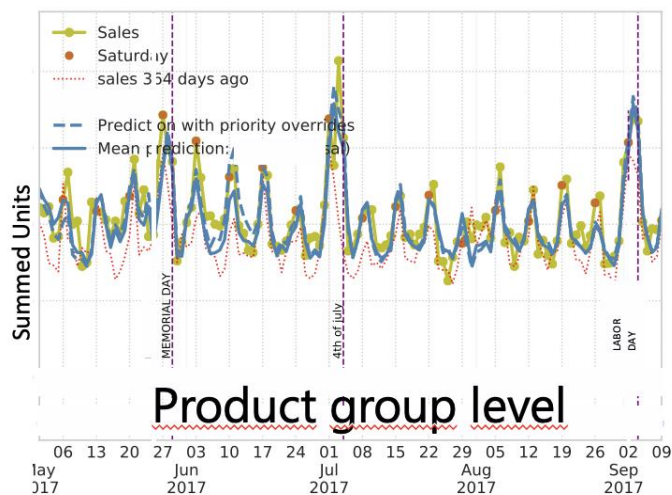
Sales
Demand Prediction



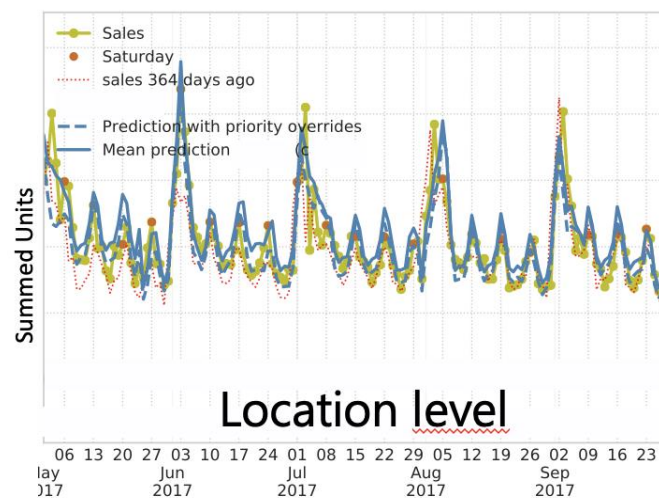
Cannibalization



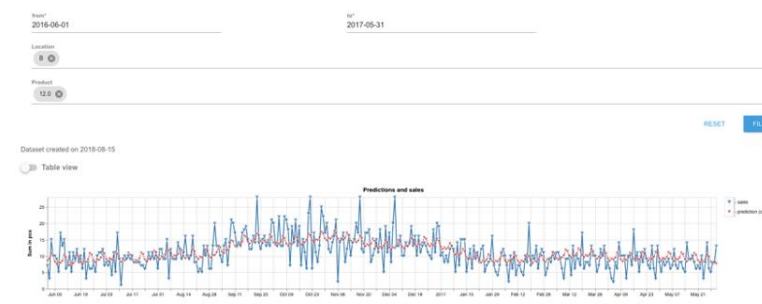
Events



Payday Effect



Seasonality

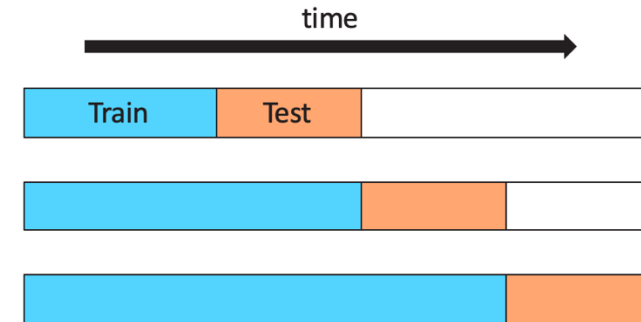
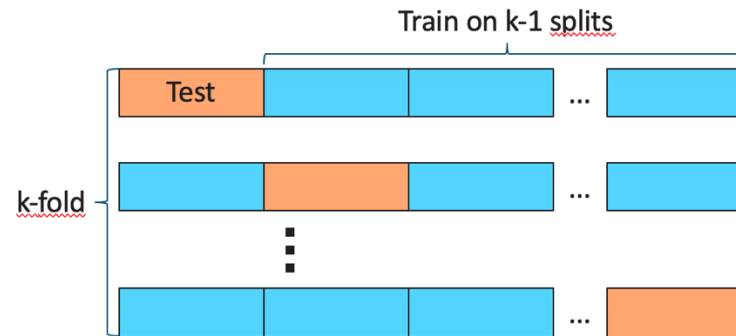


Time-Series Validation

avoid time-based leakage: only use past data at prediction time

train-test splits in time:

cross-validation:



Evaluation of Forecasts

goal: assess how well a forecasting model predicts unseen future values

choose an appropriate error metric, aggregated over an evaluation period: MAE, RMSE, MAPE

important design choice: what period to evaluate on?

- single holdout period: final 20% of the series
- rolling evaluation: repeated forecasts across multiple future windows
- forecast horizon-specific evaluation: 1-step, 5-step, 12-step ahead errors

Baseline Forecasts / Naive Models

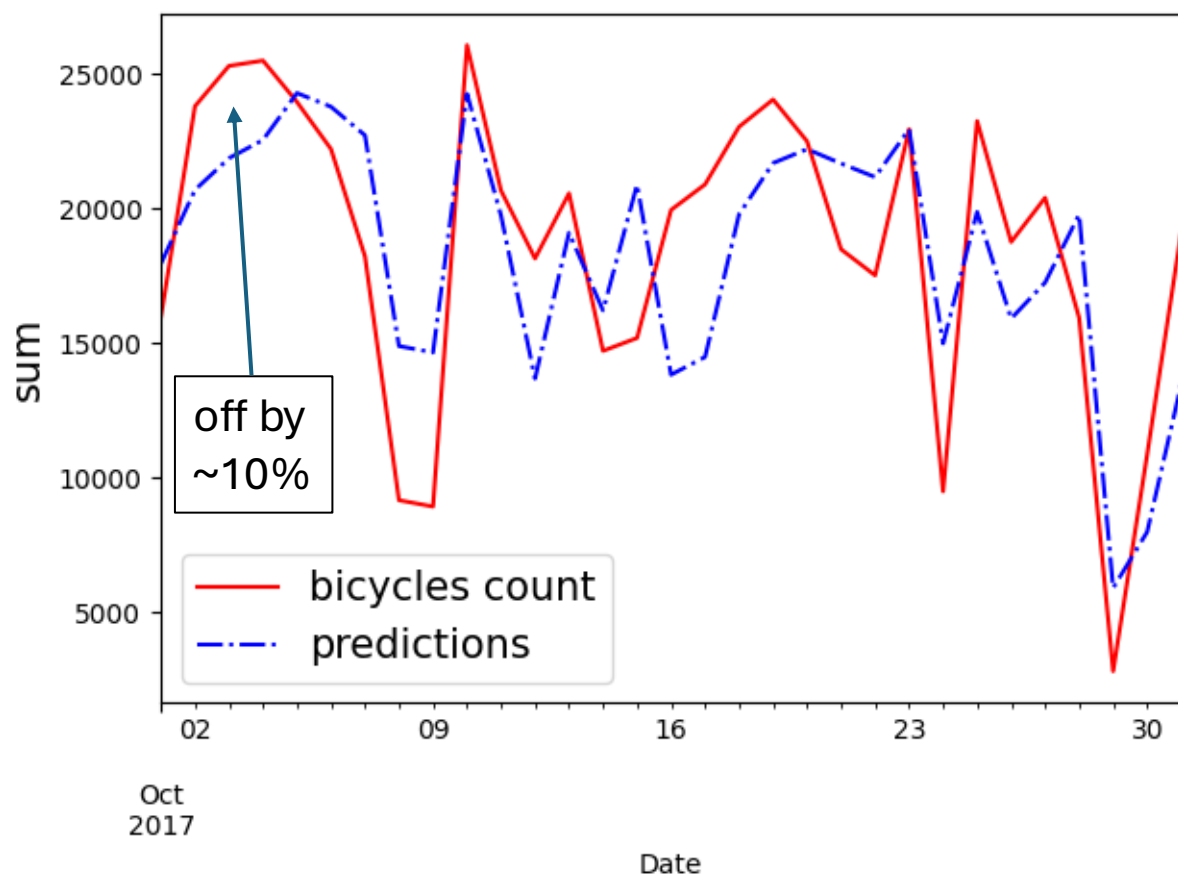
beating naive baselines matters in practice

examples:

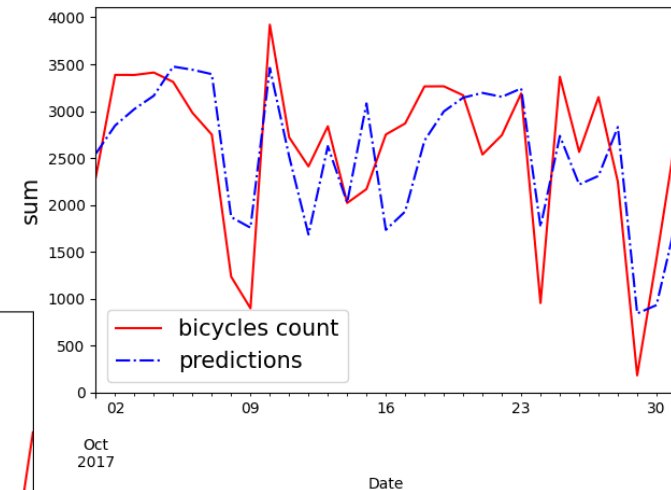
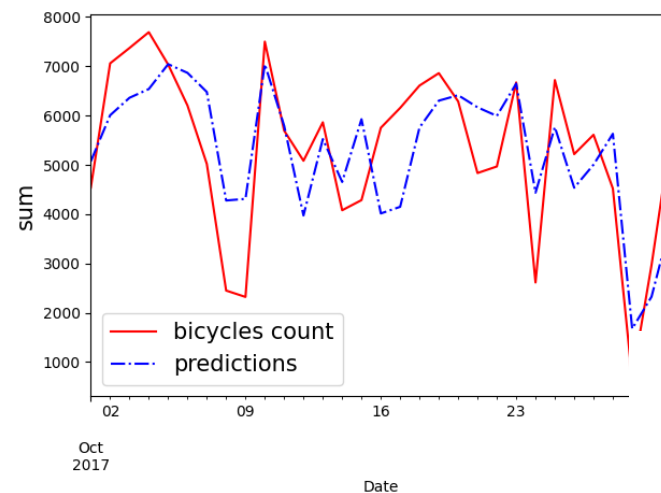
- last value forecast
- seasonal naive forecast
- moving average baseline

NY bridges bicycles count

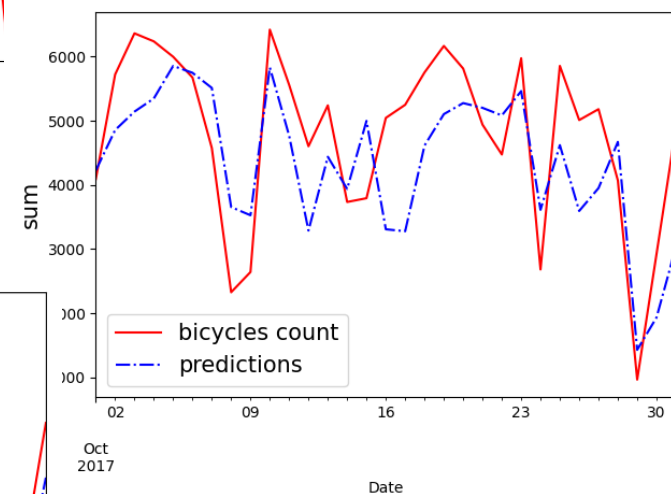
total



Manhattan bridge

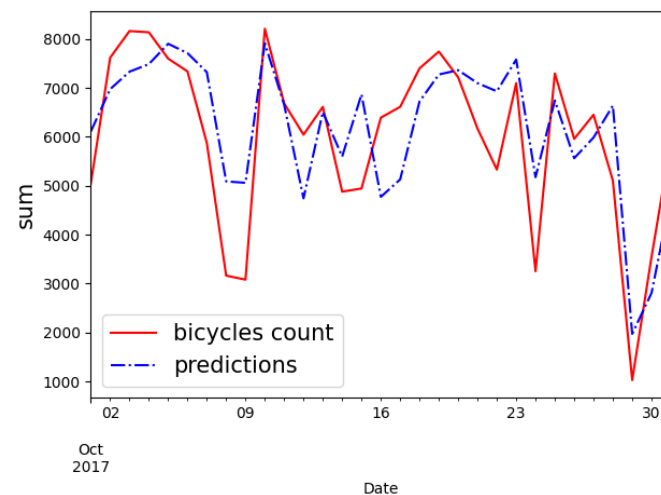


Brooklyn bridge



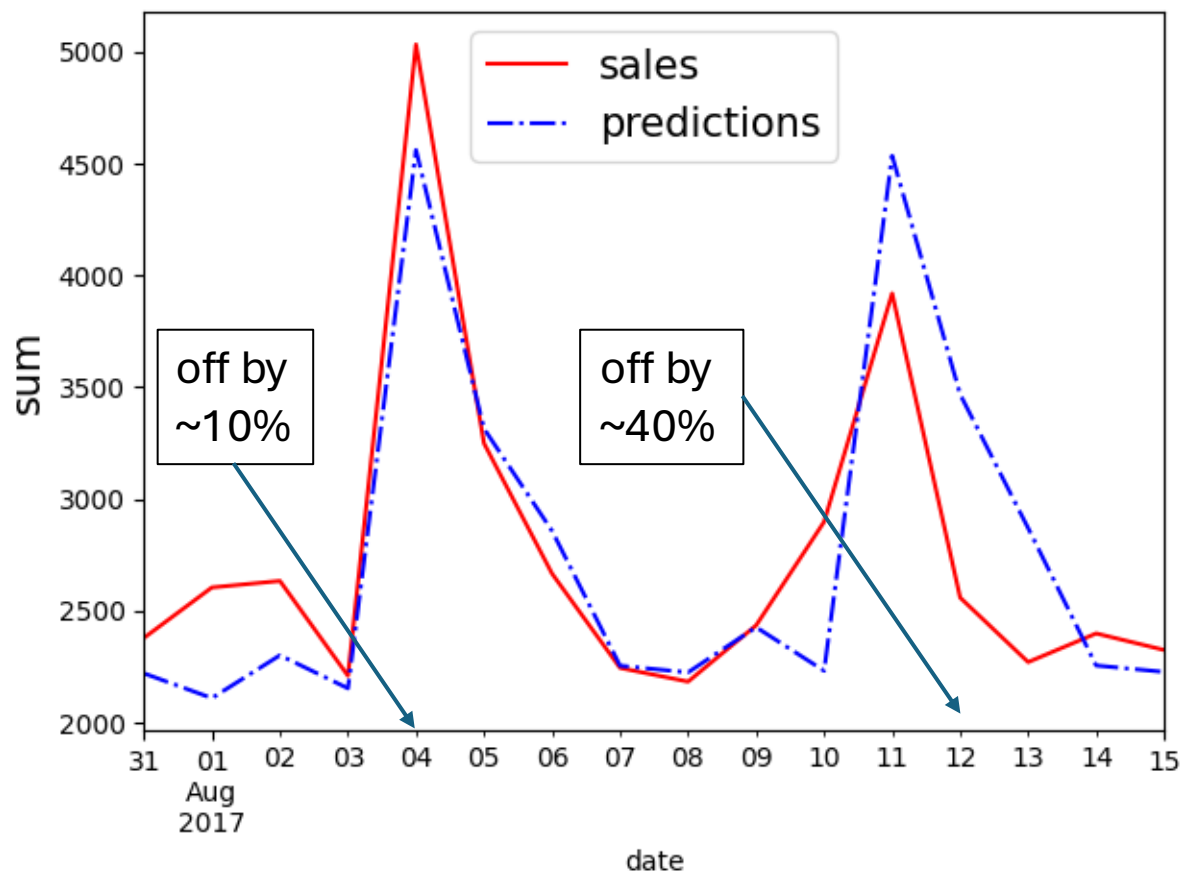
Queensboro bridge

Williamsburg bridge

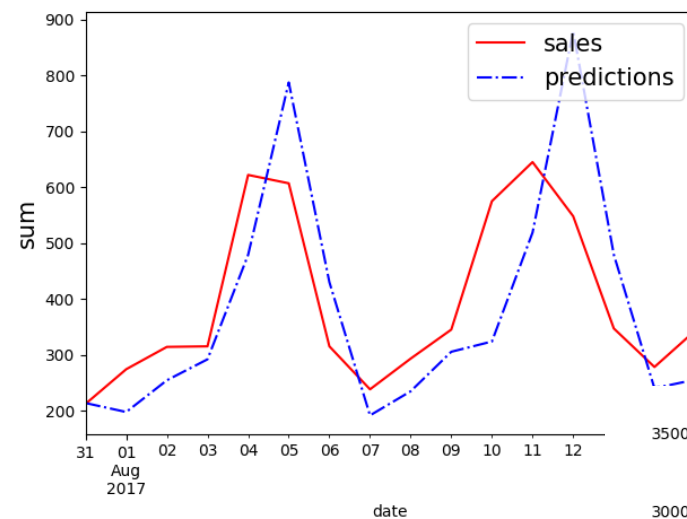


Store sales

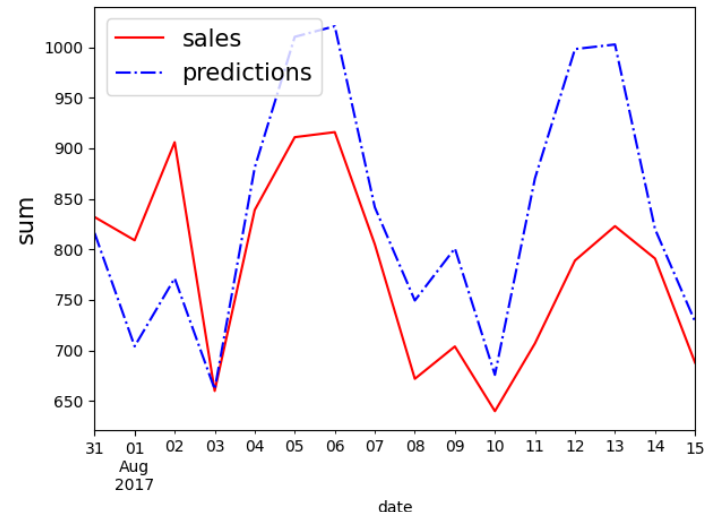
total



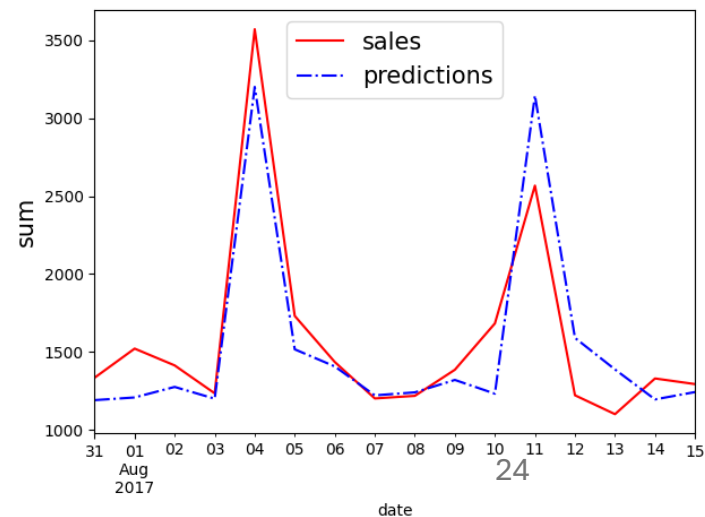
liquor, wine, beer



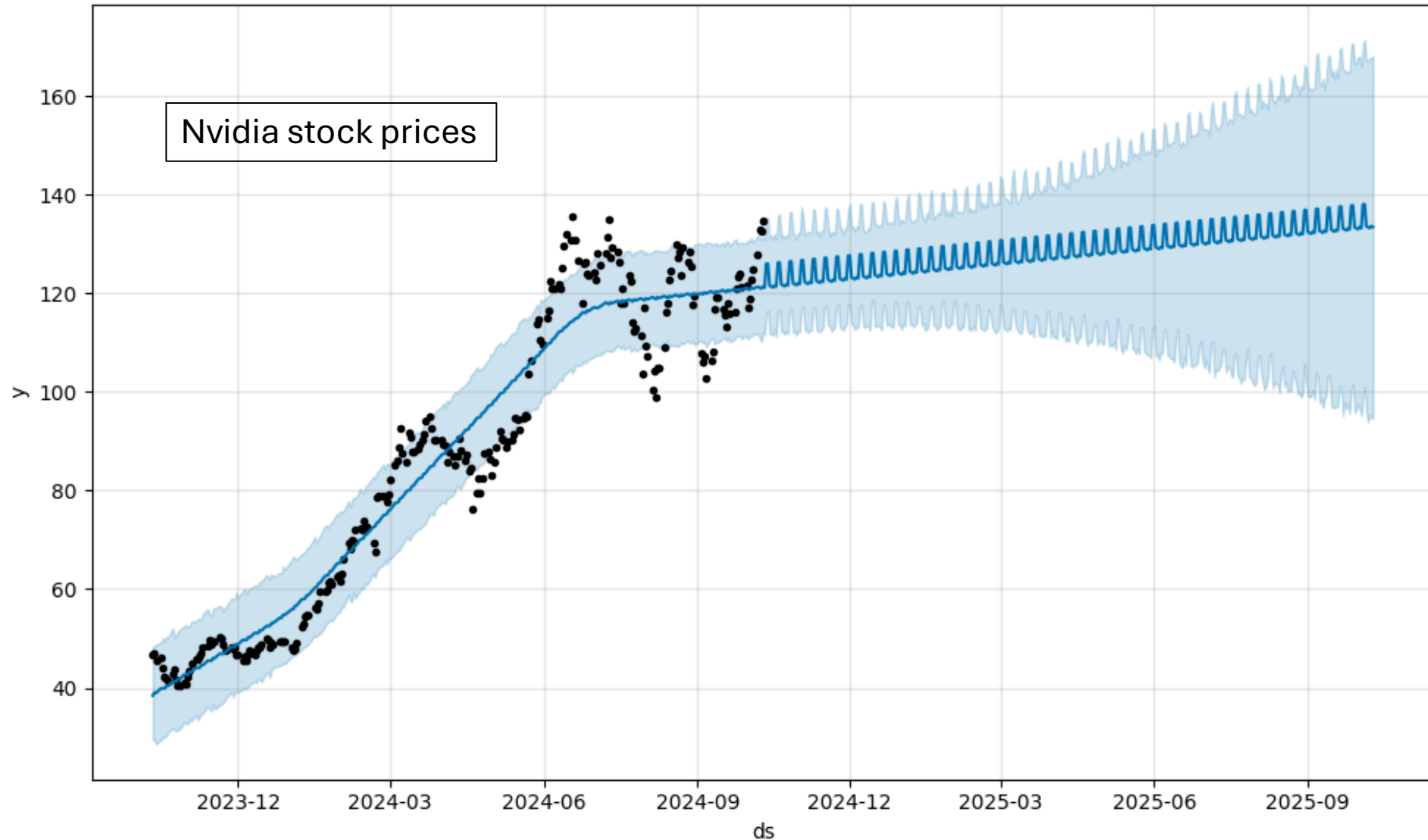
eggs



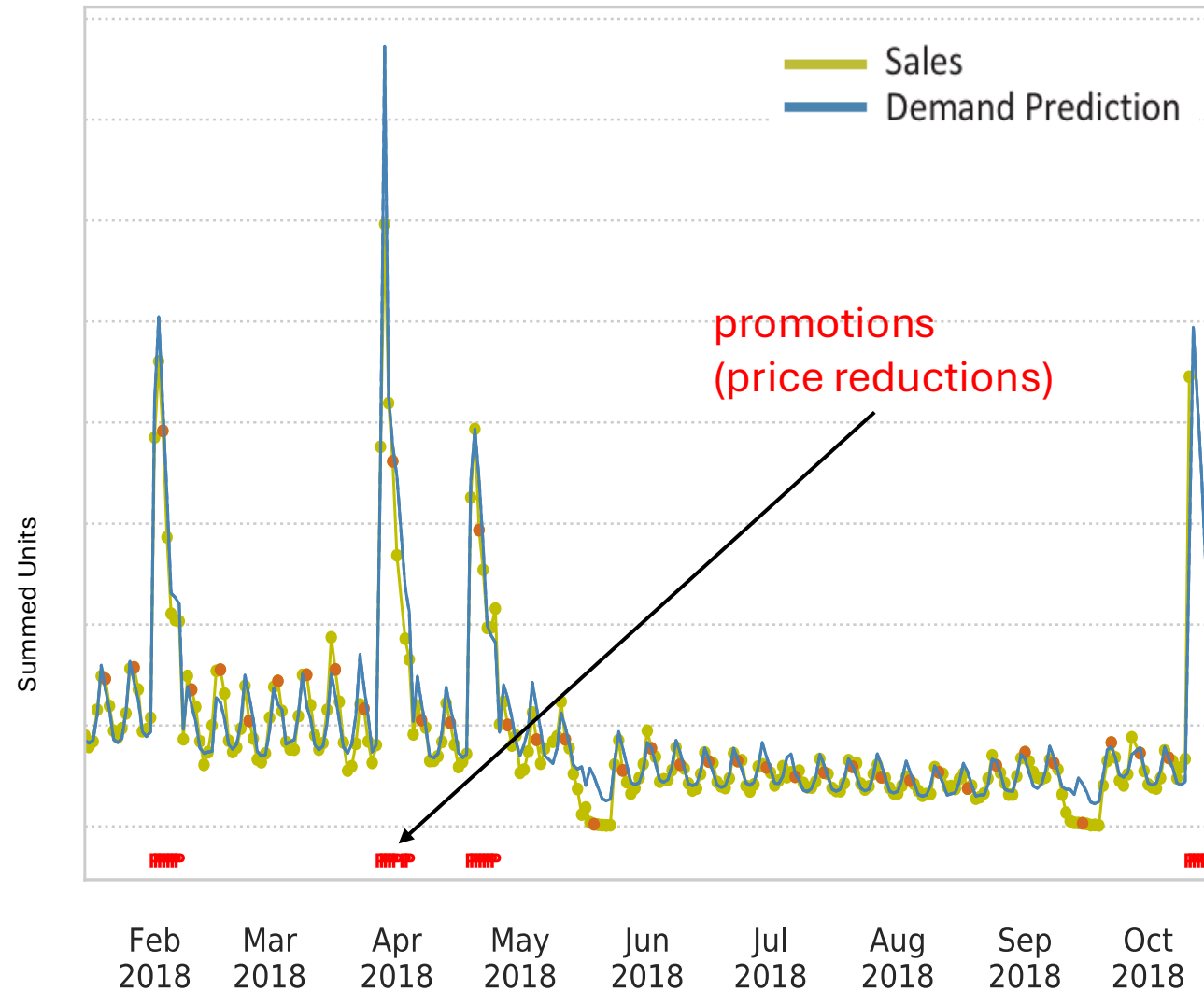
meats



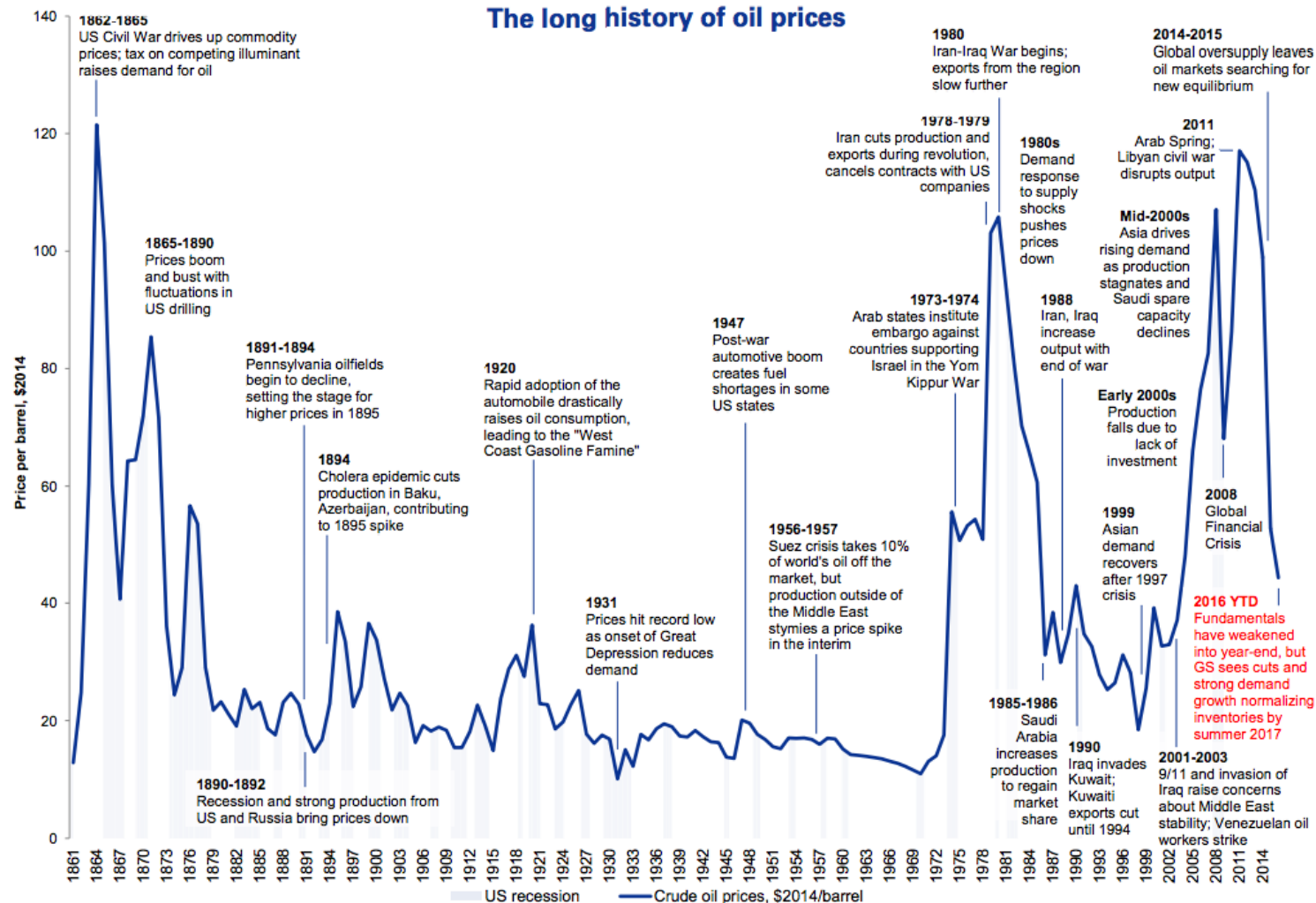
Uncertainty increases with forecast horizon.



You can predict effects of foreseeable events.



But you cannot predict effects of unforeseeable events.



An earlier version of this chart appeared on pg. 16 of *Top of Mind Issue #52: OPEC and Oil Opportunities*.

Note: 2016 price shown is YTD average as of Dec. 19, 2016.

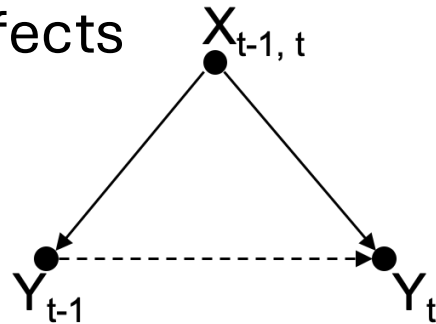
Source for data: BP, NBER/Federal Reserve Bank of St. Louis, Haver Analytics.

Source for annotations: ©James Hamilton, "Historical Oil Shocks," University of California, San Diego, February 2011; various news sources; Goldman Sachs Global Investment Research.

Temporal Confounding

auto-correlation often only spurious: common causes at consecutive times

→ no direct causal effects



specific lags such as day of week
can be interpreted both ways

advantages of learning direct causal effects from exogenous information instead of confounded auto-correlation:

- explainability
- long-term forecasting
- predictability of rare events

Beware of (Spurious) Auto-Correlation

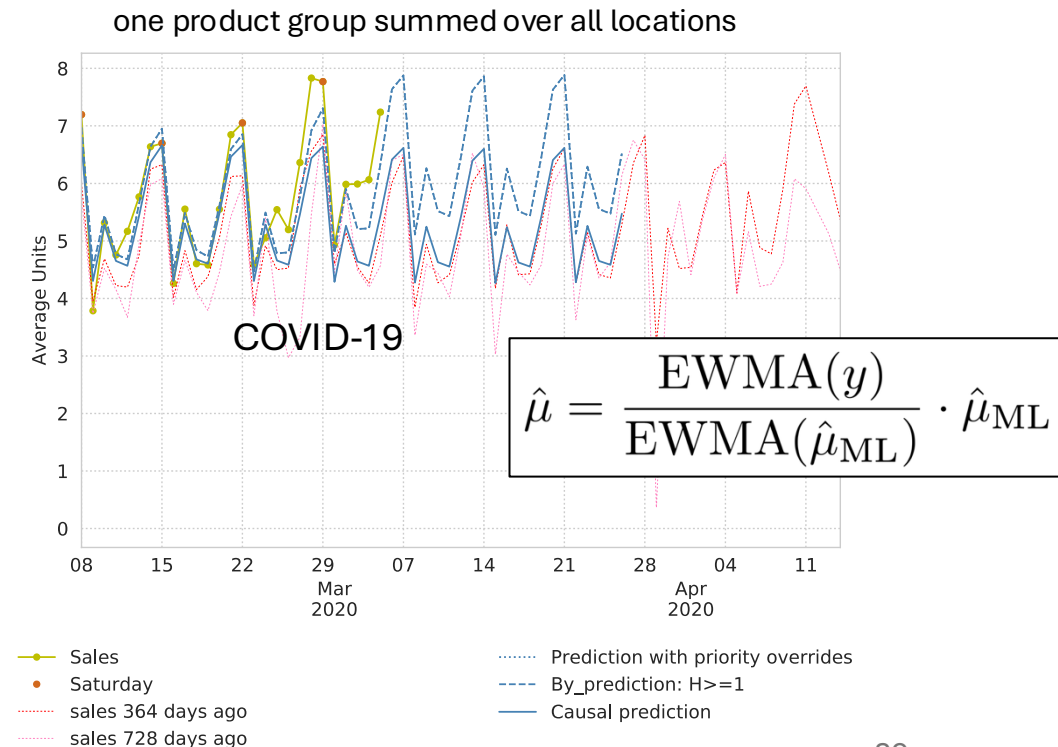
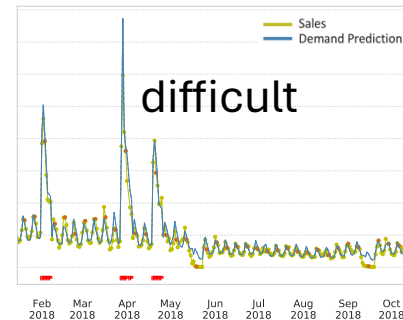
Most forecasting methods rely on lagged target information.

- univariate time series methods
- exponential smoothing features in ML model
- sequence modeling in deep learning (e.g., recurrent neural networks)

However, this makes learning of exogenous effects much harder.

- already blended in target auto-correlation
- delayed effects of short-lived structures/trends

→ simple solution: no use of target auto-correlation in ML model, but subsequent residual correction to capture missing trends



Deep Learning Methods for Sequential Data

→ direct inclusion of auto-correlation in ML model

(but beware: autocorrelation often reflects latent/exogenous drivers rather than direct temporal causation)

convolutional neural networks (CNN)

- main application computer vision: images correspond to regular 2D grids of data (pixels)
- time series data as 1D grid at regular time intervals
- parameter sharing via convolutions (and pooling) → highly regularized feed-forward networks
- fixed time window (kernel)
- multivariate time series model via different input channels

recurrent neural networks (RNN)

- main application natural language processing (NLP): sequence of words (tokens)
- time series data as time-tagged stream
- parameter sharing via recurrent nodes → context-awareness (input from past activations)
- variable-length time series
- multivariate time series model via more input nodes

transformers

- main application NLP (largely replaced RNNs), but also computer vision
- time series data as directed graph (attention in edges)
- masked self-attention to focus on relevant past time steps and positional encoding to include order of sequence
- multivariate time series model via input concatenation

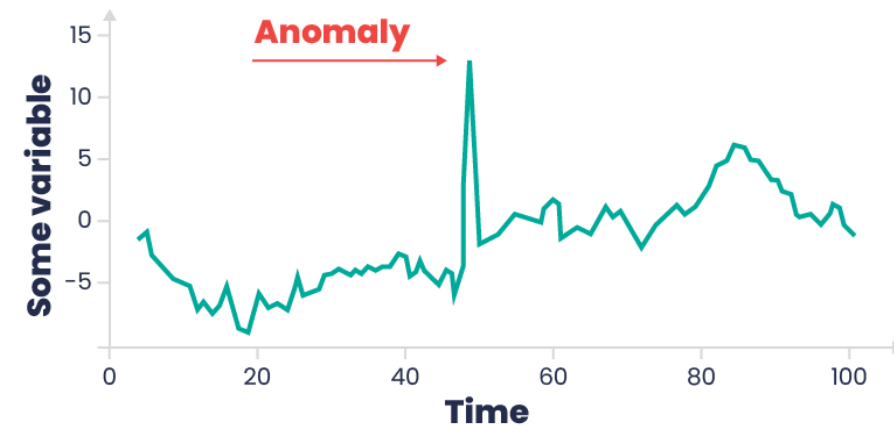
Multi-Step Forecasting Strategies

- recursive forecasting (feed predictions back)
- direct forecasting (train separate horizon-ahead models)
- multi-output forecasting

Anomaly Detection

goal: identify observations that deviate significantly from expected (temporal) behavior

- detect failures, fraud, or abnormal events
- enable early warning systems
- improve monitoring and forecasting robustness



different types:

- point anomalies: single observations that strongly differ from normal values (e.g., sudden spike in server CPU usage)
- contextual anomalies: abnormal only within a specific temporal context (e.g., high electricity demand at 3 AM)
- collective anomalies: a sequence of observations forms an unusual pattern (e.g., gradual sensor drift before equipment failure)

Anomaly Detection Approaches

methods:

- statistical: thresholds/Z-score, seasonal decomposition + residuals
- forecasting error : ARIMA , LSTM
- reconstruction error : (CNN) autoencoder
- feature-based ML: isolation forest, one-class SVM

workflow:

- preprocess data: handle missing values, detrend, deseasonalize
- model normal behavior: learn expected temporal patterns
- compute anomaly score: measure deviation from expected behavior
- flag anomalies: apply thresholds or ranking